

Programación de Servicios y Procesos

Cafetería con Threads

`wait()` y `notify()`

Autor:

Santi Martínez

December 3, 2025

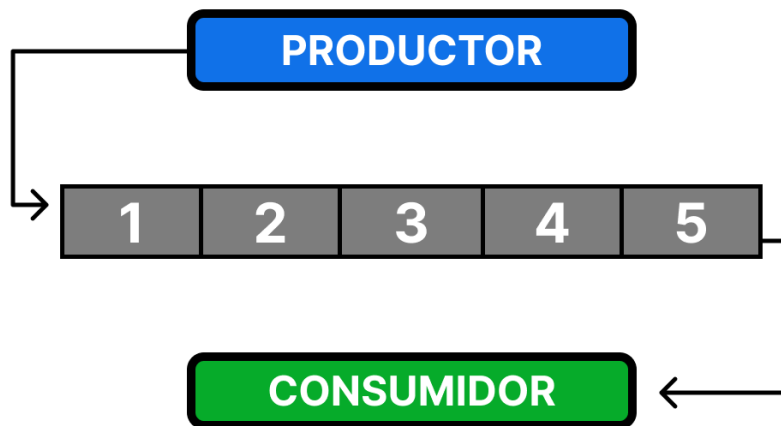
ÍNDICE

1	Introducción	2
2	Requisitos	3
2.1	Requisitos funcionales	3
2.2	Requisitos no funcionales	3
3	Modelo	4
3.1	Introducción del modelo	4
3.1.1	Buffer.java	4
3.1.2	Barista.java (Productor)	4
3.1.3	Camarero.java (Consumidor)	5

1 Introducción

Este es un programa en JavaFX en el que la principal tarea es jugar con los hilos (Threads).

La principal novedad en este programa es la implementación de un patrón productor-consumidor utilizando hilos y sincronización en Java. Para ello se han implementado los métodos **wait()** y **notify()** para gestionar el acceso concurrente a un buffer compartido entre productores (baristas) y consumidores (camareros).



En resumen, el barista produce cafés y los coloca en una mesa (buffer) con capacidad limitada. Si la mesa está llena, el barista espera hasta que un camarero recoja un café. Por otro lado, el camarero consume cafés de la mesa. Si no hay cafés disponibles, el camarero espera hasta que el barista prepare más.

Esta sincronización asegura que no haya condiciones de carrera y que los productores y consumidores operen de manera coordinada.

2 Requisitos

2.1 Requisitos funcionales

- Iniciar servicio
- Atender a clientes
- Mostrar si el cliente se fue con el café a tiempo o se fue enfadado
- Contar y mostrar los clientes que se fueron satisfechos
- Salir del programa en el momento que el usuario quiera

2.2 Requisitos no funcionales

- La interfaz se actualiza en directo
- La app es estable mientras se procesan clientes
- Funciona en cualquier plataforma que soporte Java y JavaFX



3 Modelo

3.1 Introducción del modelo

El modelo implementa el patrón productor-consumidor utilizando hilos en Java. En este caso, el barista actúa como productor que prepara cafés y los coloca en una mesa (buffer) con capacidad limitada. El camarero es el consumidor que recoge los cafés de la mesa para servirlos a los clientes.

3.1.1 Buffer.java

Clase compartida entre productores y consumidores.

- `contador_de_cafes`: Número actual de cafés en la mesa (capacidad máxima: 3)
- `put()`: Método sincronizado para que el Barista deje un café. Si la mesa está llena, espera con `wait()`
- `get()`: Método sincronizado para que el Camarero recoja un café. Si no hay cafés, espera con `wait()`
- Usa `notifyAll()` para despertar hilos en espera cuando cambia el estado del buffer

3.1.2 Barista.java (Productor)

Thread que representa al barista que prepara los cafés.

- Extiende `Thread` para ejecutarse de forma concurrente
- Realiza 35 iteraciones preparando cafés
- Llama a `buffer.put()` para dejar cada café en la mesa

- Simula tiempo de preparación aleatorio (0-500ms) con `Thread.sleep()`
- Imprime mensajes de estado en consola

3.1.3 Camarero.java (Consumidor)

Thread que representa al camarero que sirve los cafés a los clientes.

- Extiende `Thread` para ejecutarse de forma concurrente
- Realiza 35 iteraciones sirviendo cafés
- Llama a `buffer.get()` para recoger cada café de la mesa
- Simula tiempo de servicio aleatorio (0-500ms) con `Thread.sleep()`
- Imprime mensajes de estado en consola